॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

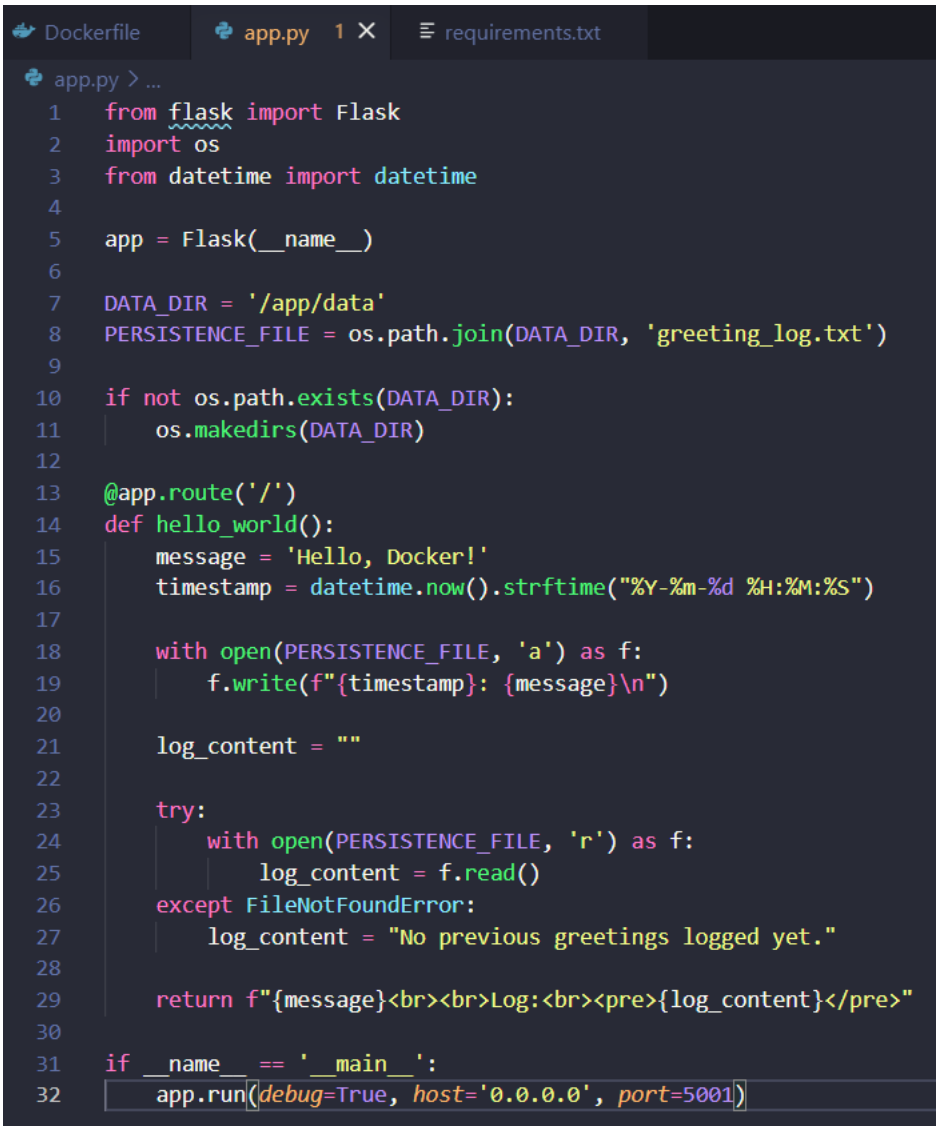
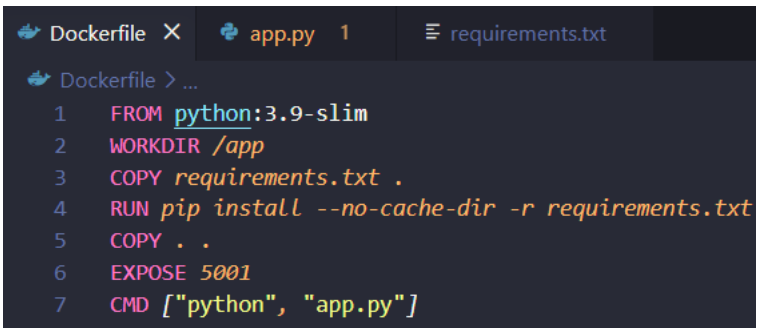
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 10

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Deepak Rathod**
PRN No.	**23070122175**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2026, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Docker Volumes & Data Persistence
Theory:	Docker volumes provide persistent storage for containers by storing data outside the container's writable layer. Unlike container storage, Docker volumes remain intact even after a container is stopped or removed, allowing data to be shared across multiple container instances. By mounting a named volume to a directory inside the container, applications can read and write data that persists independently of the container lifecycle. This makes Docker volumes ideal for storing application data, logs, databases, and other important files that should not be lost when containers are recreated. **Commands Used** • Built the updated Docker image with a new version tag: docker build -t my-flask-app:v2 . • Created a Docker named volume: docker volume create my-app-data • Started a container with the named volume mounted: docker run -d -p 8082:5001 -v my-app-data:/app/data --name flask-volume-demo my-flask-app:v2 • Stopped the running container: docker stop flask-volume-demo • Removed the stopped container: docker rm flask-volume-demo • Started a new container using the same Docker volume: docker run -d -p 8082:5001 -v my-app-data:/app/data --name flask-

	volume-demo2 my-flask-app:v2
Implementation:	**Step 1: Modify the Python Application**  Modified Flask Application — Updated the Flask application to write timestamped greetings to a log file inside the /app/data directory and configured the application to run on port 5001. **Step 2: Update the Dockerfile** 

Dockerfile | app.py 1 | requirements.txt

app.py

```python
from flask import Flask
import os
from datetime import datetime

app = Flask(__name__)

DATA_DIR = '/app/data'
PERSISTENCE_FILE = os.path.join(DATA_DIR, 'greeting_log.txt')

if not os.path.exists(DATA_DIR):
    os.makedirs(DATA_DIR)

@app.route('/')
def hello_world():
    message = 'Hello, Docker!'
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open(PERSISTENCE_FILE, 'a') as f:
        f.write(f"{timestamp}: {message}\n")

    log_content = ""

    try:
        with open(PERSISTENCE_FILE, 'r') as f:
            log_content = f.read()
    except FileNotFoundError:
        log_content = "No previous greetings logged yet."

    return f"{message}<br><br>Log:<br><pre>{log_content}</pre>"

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5001)
```

Dockerfile | app.py 1 | requirements.txt

Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5001
CMD ["python", "app.py"]
```

Updated Dockerfile — Modified the Dockerfile to expose port 5001, matching the updated Flask application configuration.

Step 3: Build the New Docker Image

```
C:\Users\dr787\Desktop\DevOps-Flask-App>docker build -t my-flask-app .
[+] Building 57.0s (11/11) FINISHED                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                        0.0s
 => => transferring dockerfile: 217B                                        0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim         9.9s
 => [auth] library/python:pull token for registry-1.docker.io               0.0s
 => [internal] load .dockerignore                                           0.1s
 => => transferring context: 2B                                             0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:dddfd7e07f9       21.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:dddfd7e07f9d       0.1s
 => => sha256:5e62c0470a0973ad9561440c8a37984c3252eda1dd1 249B / 249B       0.3s
 => => sha256:e360d85fdc0f3910a4fe48ba2bb0865d5e618 12.12MB / 12.12MB       5.1s
 => => sha256:a510a9ea6d60786b7cefdbf9577945344c72879 4.27MB / 4.27MB       4.9s
 => => sha256:6b37362b3da78869050b894b799ad4df04f1 29.83MB / 29.83MB       11.2s
 => => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774       5.8s
 => => extracting sha256:a510a9ea6d60786b7cefdbf9577945344c72879881de       0.7s
 => => extracting sha256:e360d85fdc0f3910a4fe48ba2bb0865d5e618392ea8a       3.0s
 => => extracting sha256:5e62c0470a0973ad9561440c8a37984c3252eda1dd19       0.1s
 => [internal] load build context                                           0.1s
 => => transferring context: 427B                                           0.0s
 => [2/5] WORKDIR /app                                                      0.4s
 => [3/5] COPY requirements.txt .                                           0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt               19.2s
 => [5/5] COPY app.py .                                                     0.3s
 => exporting to image                                                      4.8s
 => => exporting layers                                                     3.0s
 => => exporting manifest sha256:cea68a49be74ee2f2e4110fc1e1af5c80af3       0.0s
 => => exporting config sha256:db3920800a111d3a60deda03dd5fe08e4c0728       0.0s
 => => exporting attestation manifest sha256:11e2637afc12f52b9f85d3d5       0.0s
 => => exporting manifest list sha256:35163a9e0369bf4a5cd558e633c3450       0.0s
 => => naming to docker.io/library/my-flask-app:latest                      0.1s
 => => unpacking to docker.io/library/my-flask-app:latest                   1.3s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/dt1446433onk0yqgzkzn5yj2j

C:\Users\dr787\Desktop\DevOps-Flask-App>docker run -d --name my-flask-app -p 5000:5000 my-flask-app
a4dfbddac0bdc5f54261a2390cb09a99408c3c025010078ecc2aa97e42c0ce3f
```

Built Updated Docker Image — Built a new Docker image named my-flask-app:v2 containing the modified Flask application configured for Docker volume-based data persistence.

Step 4: Create a Docker Named Volume

```
C:\Users\dr787\Desktop\DevOps-Flask-App>
C:\Users\dr787\Desktop\DevOps-Flask-App>docker volume create flask-volume
flask-volume

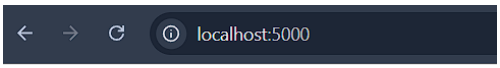
C:\Users\dr787\Desktop\DevOps-Flask-App>docker run -d --name my-flask-app-volume -p 5001:5000 -v flask-volume:/data my-flask-app
6da57b9bd80c9619650dee3deb7a81f96434f1b97255db02e8b9eb9b6a432340

C:\Users\dr787\Desktop\DevOps-Flask-App>docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS                    PORTS
                                  NAMES
6da57b9bd80c   my-flask-app           "python app.py"          6 seconds ago    Up 6 seconds              0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
                                  my-flask-app-volume
a4dfbddac0bd   my-flask-app           "python app.py"          6 minutes ago    Up 6 minutes              0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
                                  my-flask-app
b95fb0f41b30   ubuntu                 "/bin/bash"              22 minutes ago   Up 22 minutes
                                  my-flask-app-new
cadc2c25ac67   jenkins/jenkins:lts    "/usr/bin/tini -- /u…"   46 minutes ago   Up 46 minutes             0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0
.0.0:50000->50000/tcp, [::]:50000->50000/tcp   jenkins
fd007f42c00c   lumen-backend-fastapi  "/usr/bin/tini -- uv…"   3 weeks ago      Up 56 minutes (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
                                  lumen_fastapi
29a33b4280fa   redis:7-alpine         "docker-entrypoint.s…"   4 weeks ago      Up 56 minutes (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
                                  lumen_redis
36b70b46b803   postgis/postgis:17-3.5 "docker-entrypoint.s…"   4 weeks ago      Up 56 minutes (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
                                  lumen_postgres
```

Created Docker Volume — Created a Docker named volume called my-app-data to store application data independently of the container, ensuring data persistence across container restarts and removal.

Step 5: Access the Application and Write Data

```
C:\Users\dr787\Desktop\DevOps-Flask-App>docker run -d --name my-flask-app-volume -p 5001:5000 -v flask-volume:/data my-flask-app
6da57b9bd80c9619650dee3deb7a81f96434f1b97255db02e8b9eb9b6a432340
```



My Flask Docker App

Flask application successfully running inside Docker.

DevOps Lab Assignment - Docker Containerization

Verified Data Logging — Accessed the Flask application through http://localhost:8082 and refreshed the page multiple times to verify that timestamped greetings were successfully written to the persistent log file stored in the Docker volume.

Step 6: Stop and Remove the Container

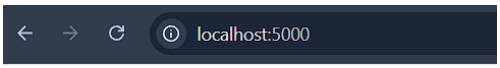
```
C:\Users\dr787\Desktop\DevOps-Flask-App>docker stop my-flask-app-volume
my-flask-app-volume

C:\Users\dr787\Desktop\DevOps-Flask-App>docker rm my-flask-app-volume
my-flask-app-volume

C:\Users\dr787\Desktop\DevOps-Flask-App>
```

Stopped and Removed Container — Stopped the flask-volume-demo container and removed it using the docker stop and docker rm commands while preserving the application data stored in the Docker named volume.

Step 7: Verify Data Persistence



My Flask Docker App

Flask application successfully running inside Docker.

DevOps Lab Assignment - Docker Containerization

Verified Data Persistence — Accessed the Flask application after starting a new container with the same Docker named volume and confirmed that the previously stored log entries were retained. This demonstrated successful data persistence across multiple container instances.

Subject Faculty Name: Prof. Arijit Dutta